OOP Project – Subtask 1

Project:

Aura Retail OS

Submitted by – Prime Coders

(Durgesh, Priyanshu, Moksh & Jay)

Date – 29 March 2026

High-Level Architecture Diagram

Aura Retail OS is designed as a modular, extensible system that decouples hardware, payment systems, and inventory management from core kiosk logic.

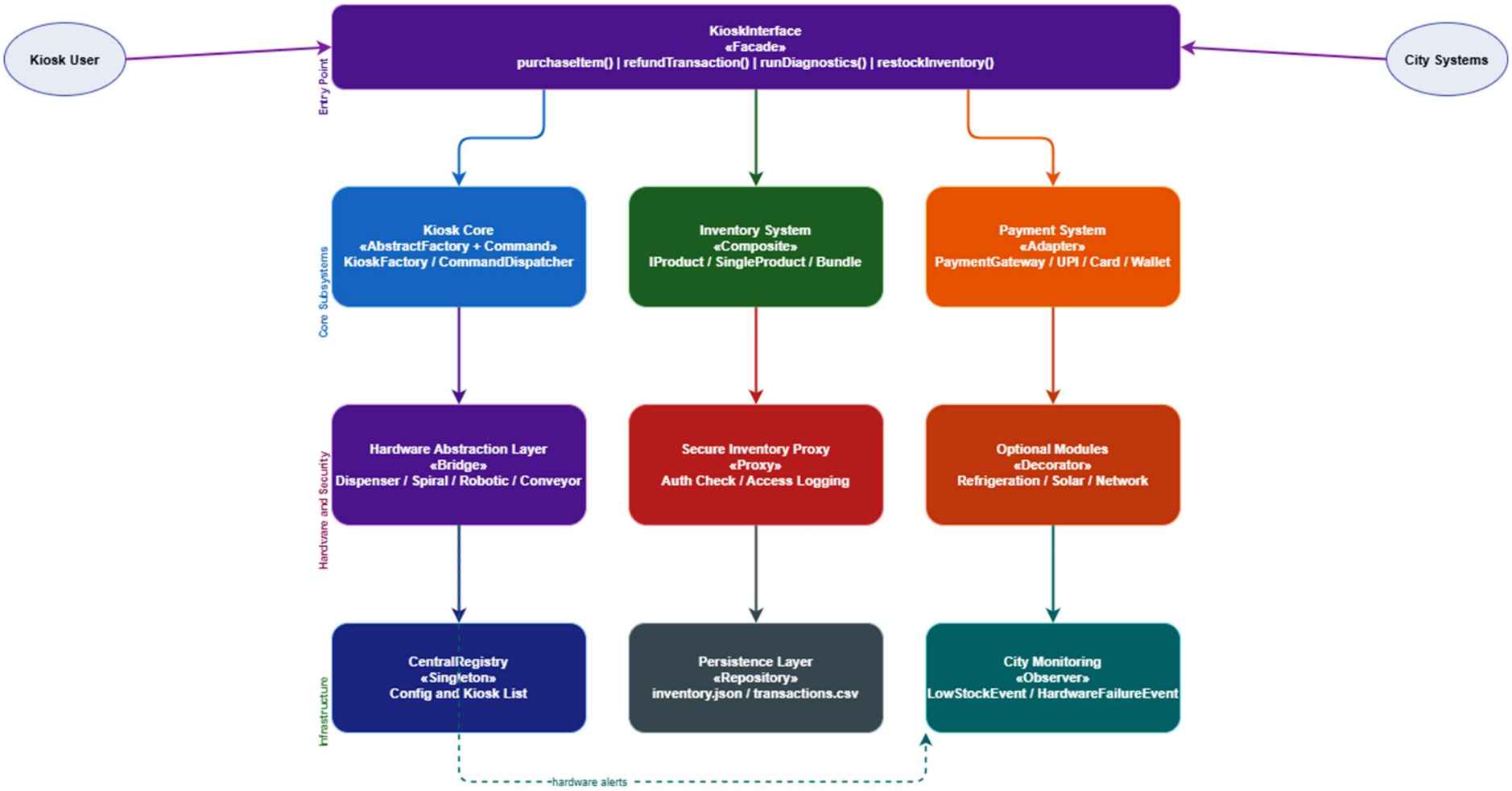
The architecture is divided into independent subsystems that interact through well-defined interfaces to ensure low coupling and high cohesion.

Key architectural goals:

- Hardware independence via abstraction layer
- Easy integration of new payment providers
- Modular inventory handling with support for nested bundles
- Secure and controlled access to system components
- Scalability across multiple kiosk environments

The system follows a layered architecture where the Kiosk Core System orchestrates operations while delegating responsibilities to specialized subsystems.

Aura Retail OS — High-Level Architecture



Preliminary Class Diagram

The class design of Aura Retail OS follows object-oriented principles such as encapsulation, abstraction, inheritance, and polymorphism.

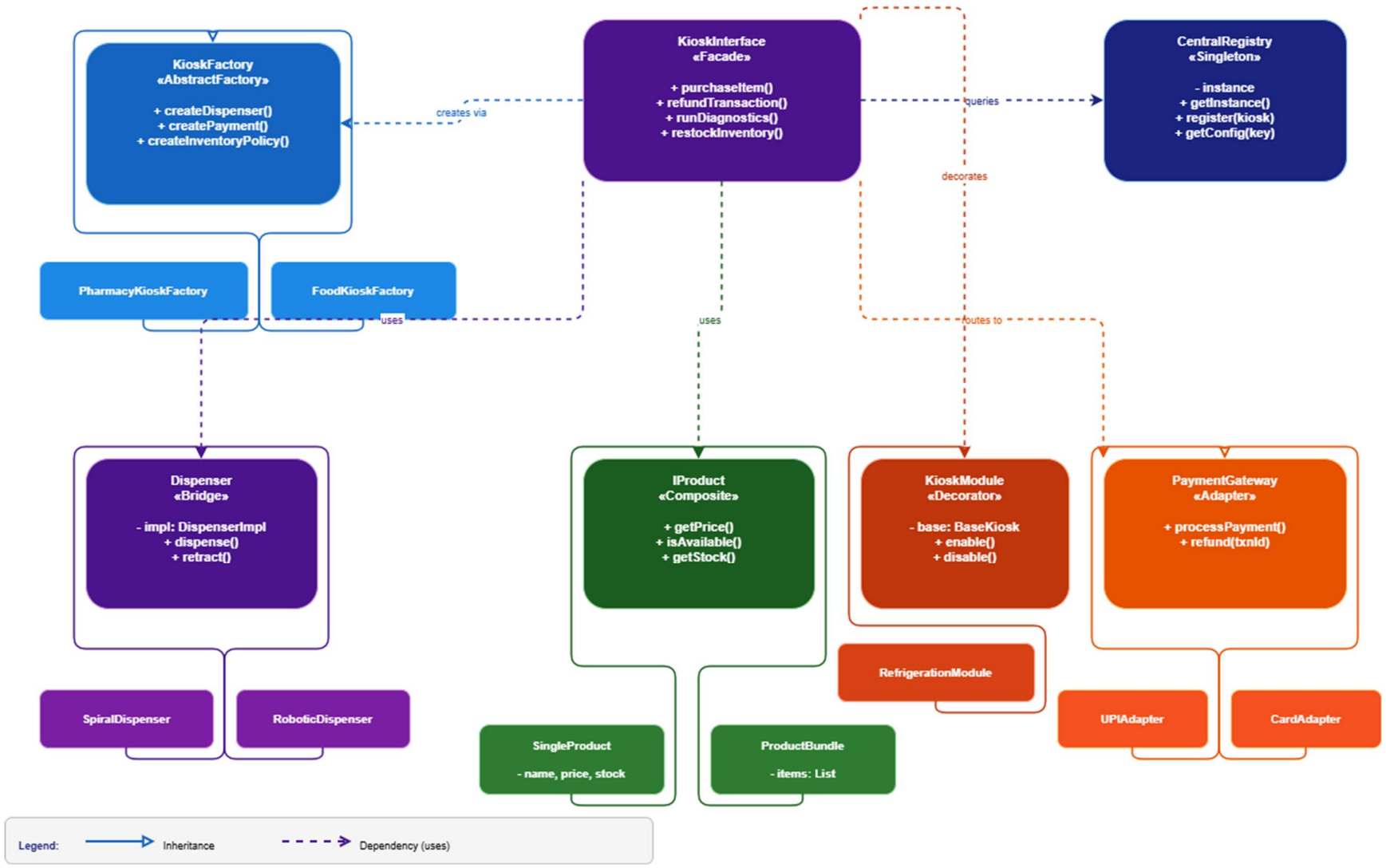
Interfaces are heavily used to decouple implementations from usage, allowing runtime flexibility in hardware modules, payment providers, and pricing strategies.

Design considerations:

- Use of interfaces for extensibility
- Composition over inheritance where applicable
- Clear separation of responsibilities
- Support for dynamic module addition

The design ensures that new hardware types, payment methods, and inventory structures can be integrated without modifying existing core logic.

Aura Retail OS — Preliminary Class Diagram (Path B)



Planned Design Patterns

1. Singleton Pattern

Where: CentralRegistry

Problem it solves: The system needs one central place to store kiosk configuration, registered kiosks, and system status. If multiple instances existed, they would hold conflicting data.

How we use it: CentralRegistry restricts instantiation to a single object using a private constructor and a static getInstance() method. All subsystems access configuration and kiosk registration through this one instance.

Key classes: CentralRegistry

2. Factory Method Pattern

Where: Kiosk creation — PharmacyKioskFactory, FoodKioskFactory, EmergencyReliefKioskFactory

Problem it solves: The system needs to create different types of kiosks without exposing the construction logic to the rest of the system. Using new directly throughout the code creates tight coupling.

How we use it: Each kiosk type has its own factory class. The client simply calls the factory and gets back a fully constructed kiosk object without knowing the internal details.

Key classes: KioskCreator, PharmacyKioskFactory, FoodKioskFactory, EmergencyReliefKioskFactory

3. Abstract Factory Pattern

Where: Component creation — KioskFactory

Problem it solves: Each kiosk type needs a compatible set of components — the right dispenser, the right payment module, and the right inventory policy. Creating them independently risks mismatched combinations.

How we use it: KioskFactory is an abstract class that defines methods like createDispenser(), createPaymentModule(), and createInventoryPolicy(). Each concrete factory (e.g. PharmacyKioskFactory) overrides these to return components that are guaranteed to work together.

Key classes: KioskFactory, PharmacyKioskFactory, FoodKioskFactory, EmergencyReliefKioskFactory

4. Adapter Pattern

Where: Payment system — UPIAdapter, CardAdapter, WalletAdapter

Problem it solves: Different payment providers (UPI, credit card, digital wallet) all have incompatible APIs. The core system cannot be rewritten every time a new payment provider is added.

How we use it: We define a common PaymentGateway interface with processPayment() and refund() methods. Each third-party payment API is wrapped inside an adapter class that translates its specific calls into this common interface. Adding a new provider only requires writing one new adapter class.

Key classes: PaymentGateway, UPIAdapter, CardAdapter, WalletAdapter

5. Composite Pattern

Where: Inventory system — IProduct, SingleProduct, ProductBundle

Problem it solves: The inventory contains both individual products and bundles (e.g. emergency kits containing multiple products or even other bundles). Operating on them differently throughout the codebase makes the code complex and fragile.

How we use it: Both SingleProduct and ProductBundle implement the same IProduct interface. When getPrice() or isAvailable() is called on a bundle, it recursively calls the same method on all its children. The rest of the system treats a bundle exactly like a single product.

Key classes: IProduct, SingleProduct, ProductBundle

6. Facade Pattern

Where: System entry point — KioskInterface

Problem it solves: The system has multiple subsystems (inventory, payment, hardware, registry). External systems should not need to interact with all of them directly as this creates high coupling.

How we use it: KioskInterface exposes only four simple operations — purchaseItem(), refundTransaction(), runDiagnostics(), and restockInventory(). Internally it coordinates across all subsystems. External code only ever talks to this one class.

Key classes: KioskInterface

Subsystem Responsibilities of each member

Name of Member	Responsibility in project
Durgesh	Hardware Abstraction Layer — Dispenser class design, SpiralDispenser and RoboticDispenser implementations, hardware swap logic, HAL documentation
Priyanshu	Inventory System — IProduct interface, SingleProduct and ProductBundle (Composite pattern), stock and price computation, inventory JSON persistence
Jay	Payment System — PaymentGateway interface, UPIAdapter, CardAdapter and WalletAdapter implementations, transaction logic
Moksh	Core System — KioskInterface (Facade), KioskFactory and concrete factories (Abstract Factory), CentralRegistry (Singleton), system configuration